Backtracking

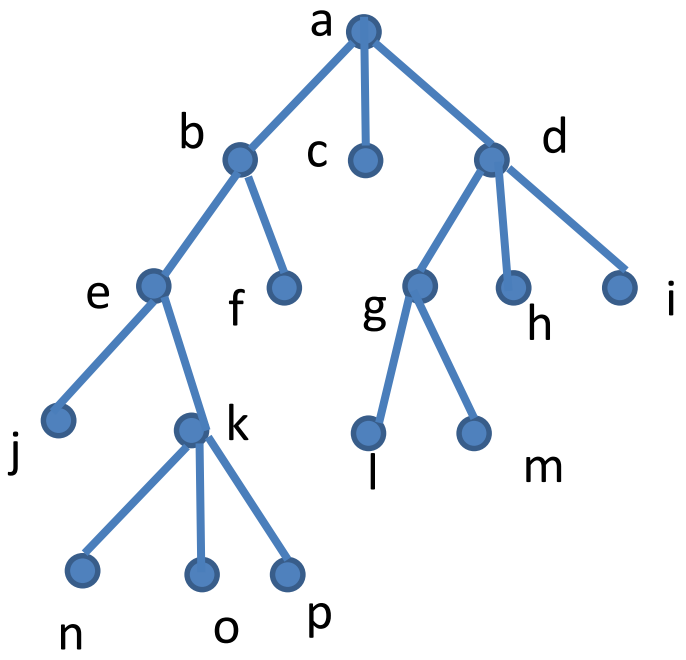
Traversal Algorithms

- preorder
- inorder
- postorder

Cr: Prof. David Matuszek

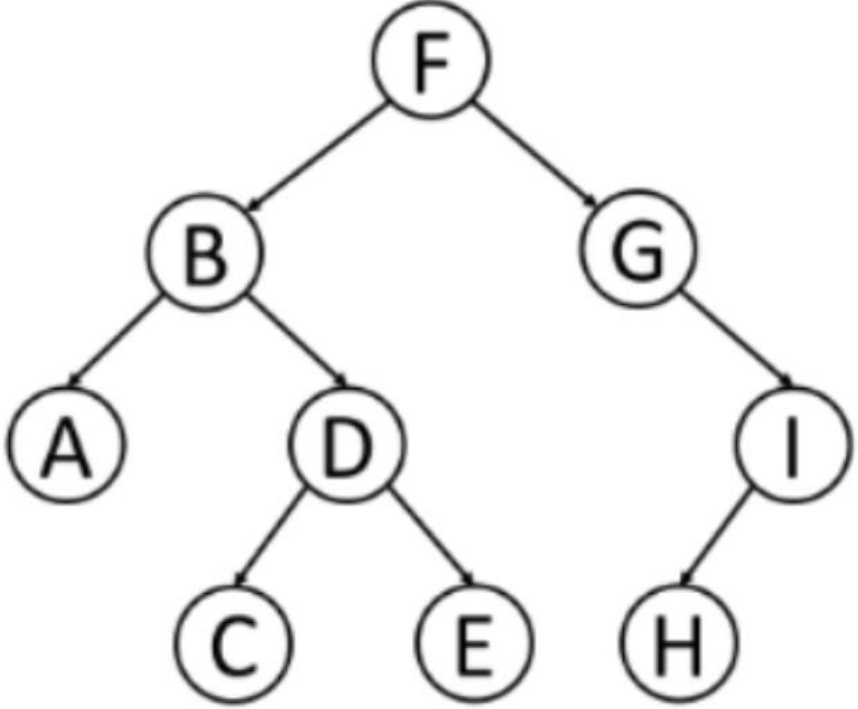
www.cis.upenn.edu/~matuszek

In which order does a preorder traversal visit the vertices in this ordered rooted tree?



procedure *preorder*(*T*: ordered rooted tree)
r := root of *T*
list *r*
for each child *c* of *r* from left to right
begin
  T(*c*) := subtree with *c* as its root
  preorder(*T*(*c*))
end

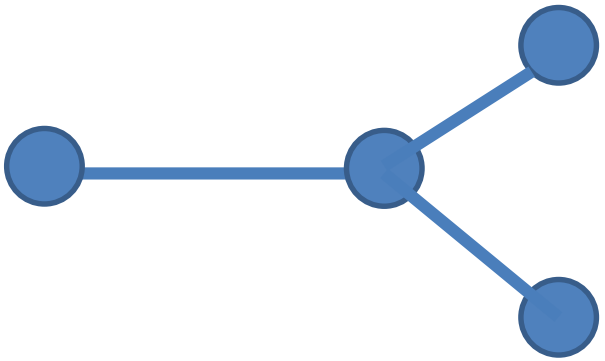
output: a b e j k n o p f c d g l m h i



- preorder F B A D C E G H
- inorder A B C E D F G H I
- postorder A C E D B H I G F

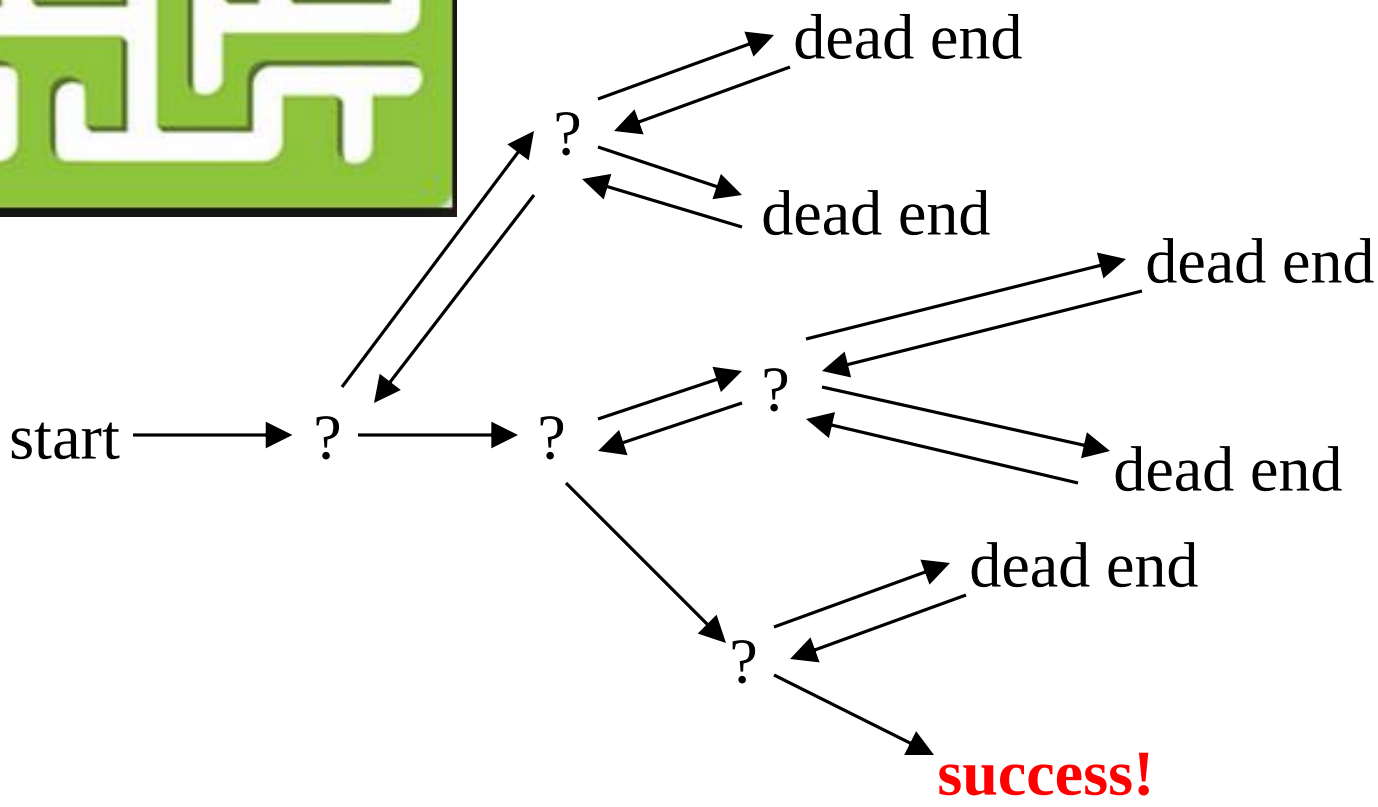
Backtracking

- วิธีการค้นหาคำตอบที่เราต้องการจากจุดตั้งต้น
- โดยที่ลักษณะของปัญหาจะมีทางแยกที่ต้องค้นหลายทาง
- เมื่อเจอทางแยก ก็จะเดินไปจนสุดเส้นทางที่ 1
  - ถ้ายังไม่พบคำตอบ ก็จะย้อนกลับมาที่ทางแยกก่อนหน้านี้ แล้วเดินไปตามเส้นทางที่ 2
  - จะค้นหาแบบนี้ไปเรื่อย ๆ จะพบเป้าหมาย หรือไปครบทุกเส้นทาง



Backtracking (animation)



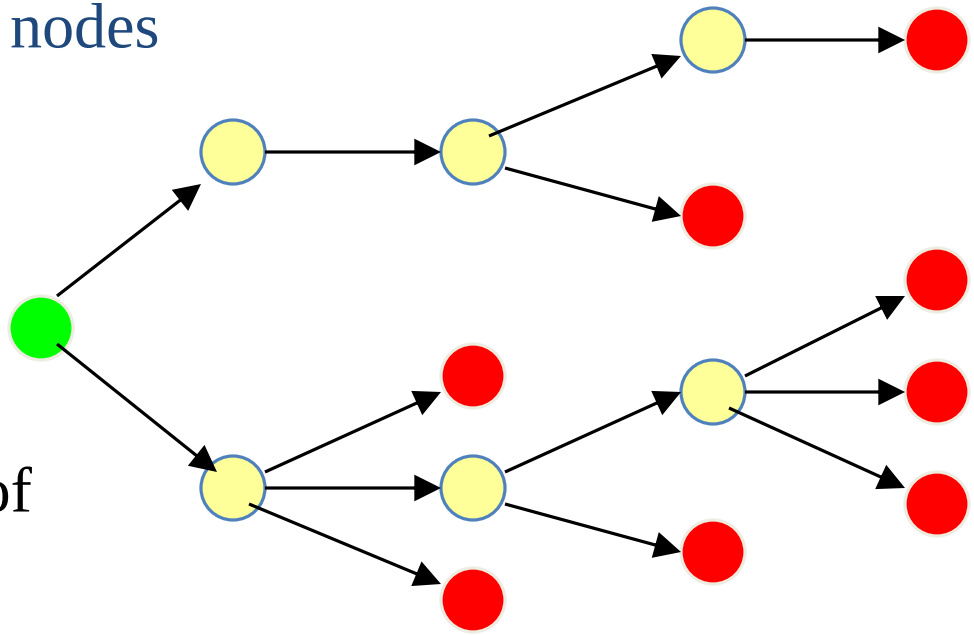


ที่มา **Prof. David Matuszek**

www.cis.upenn.edu/~matuszek

Terminology I

A tree is composed of nodes



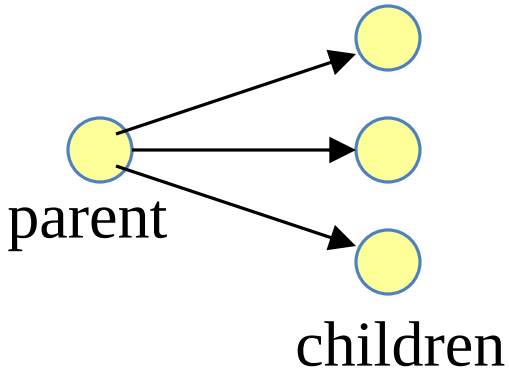
There are three kinds of nodes:

- The (one) root node
- Internal nodes
- Leaf nodes

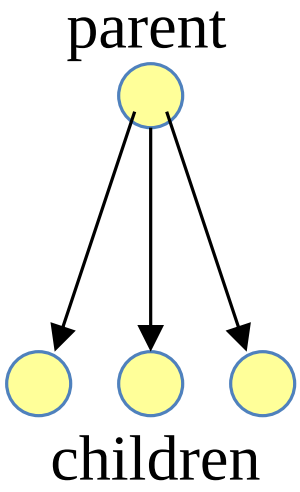
Backtracking can be thought of as searching a tree for a particular "**goal**" leaf node

Terminology II

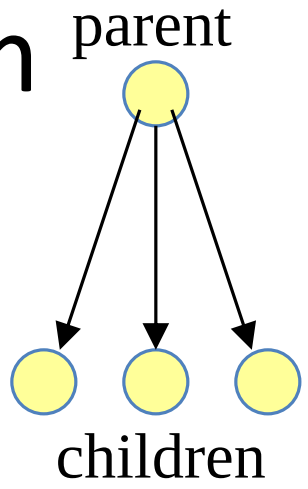
- Each non-leaf node in a tree is a parent of one or more other nodes (its children)
- Each node in the tree, other than the root, has exactly one parent



Usually, however, we draw our trees *downward*, with the root at the top



The backtracking algorithm



- Backtracking is really quite simple
- We "explore" each node, as follows:
- To "explore" node N:
    1. If N is a goal node, return "success"
    2. If N is a leaf node, return "failure"
    3. For each child C of N,
        3.1. Explore C
            3.1.1. If C was successful, return "success"
    4. Return "failure"

ปัญหาเขาวงกต

- หาเส้นทางจากจุดเริ่มต้นไปยังจุดสิ้นสุด
- โดยในแต่ละแยก เรามีทางเลือกดังนี้
  - Go straight
  - Go left
  - Go right
- เราไม่มีข้อมูลอื่นๆ เพิ่มเติม ยกเว้น ถ้าเราได้ผ่านจุดนั้น ๆ มาแล้ว



ที่มา software.thaiware.com

A More Concrete Example

- Sudoku
- 9 by 9 matrix with some numbers filled in
- all numbers must be between 1 and 9
- Goal: Each row, each column, and each mini matrix must contain the numbers between 1 and 9 once each
  - no duplicates in rows, columns, or mini matrices

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Cr: https://www.cs.utexas.edu/~scottm/cs314/handouts/slides/Topic13RecursiveBacktracking.pdf

Solving Sudoku – Brute Force

- A *brute force* algorithm is a simple but general approach
- Try all combinations until you find one that works
- This approach isn't clever, but computers are fast
- Then try and improve on the brute force results

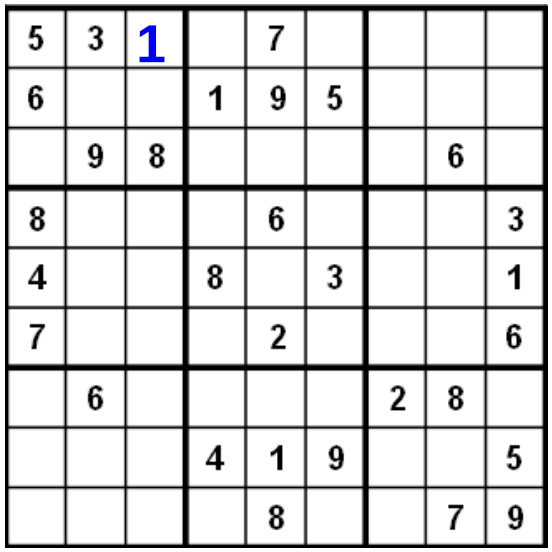
5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

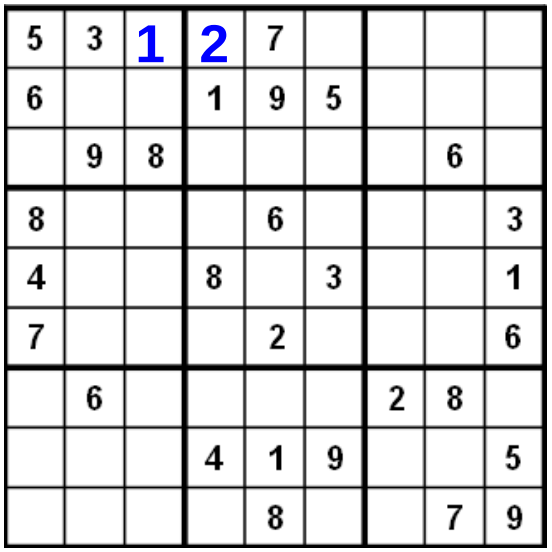
Solving Sudoku

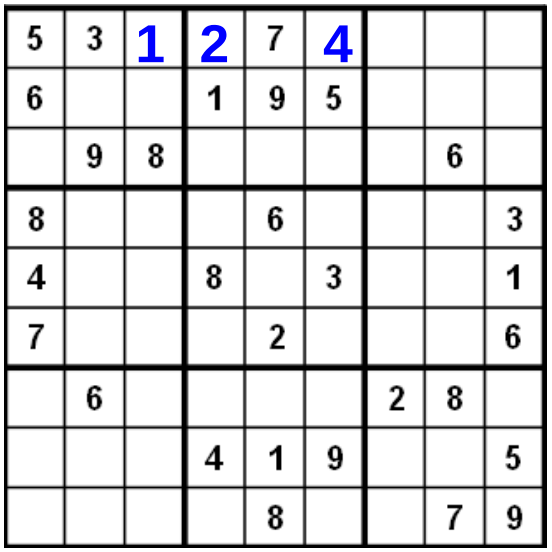
- Brute force Sudoku Solution
  - if not open cells, solved
  - scan cells from left to right, top to bottom for first open cell
  - When an open cell is found start cycling through digits 1 to 9.
  - When a digit is placed check that the set up is legal
  - now solve the board

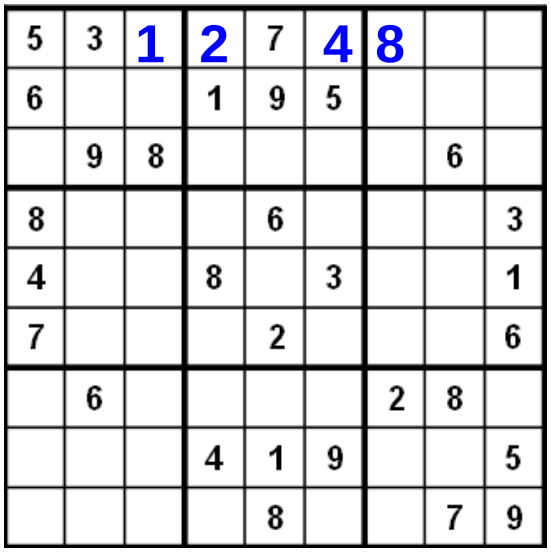
5	3	1		7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

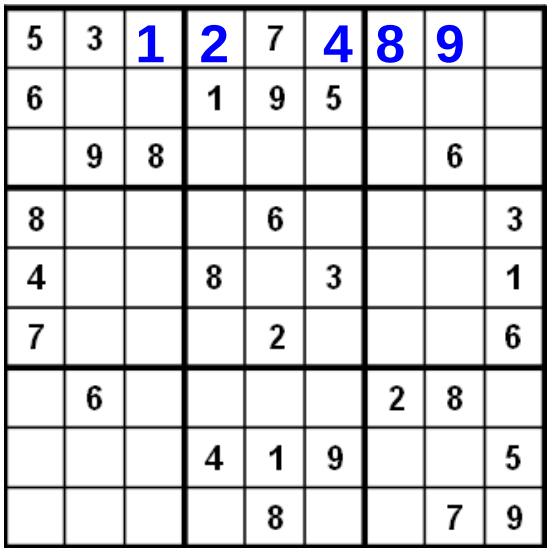
Solving Sudoku – Later Steps











uh oh!

Sudoku – A Dead End

- We have reached a dead end in our search

5	3	1	2	7	4	8	9	
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

- With the current set up none of the nine digits work in the top right corner

Backing Up

- When the search reaches a dead end in ***backs up*** to the previous cell it was trying to fill and goes onto to the next digit
- We would back up to the cell with a 9 and that turns out to be a dead end as well so we back up again
  - so the algorithm needs to remember what digit to try next
- Now in the cell with the 8. We try and 9 and move forward again.

5	3	1	2	7	4	8	9	
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

5	3	1	2	7	4	9		
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Characteristics of Brute Force and Backtracking

- Brute force algorithms are slow
- The don't employ a lot of logic
  - For example we know a 6 can't go in the last 3 columns of the first row, but the brute force algorithm will plow ahead any way
- But, brute force algorithms are fairly easy to implement as a first pass solution
  - many backtracking algorithms are brute force algorithms

Key Insights

- After trying placing a digit in a cell we want to solve the new sudoku board
  - Isn't that a smaller (or simpler version) of the same problem we started with?!?!?!?
- After placing a number in a cell the we need to remember the next number to try in case things don't work out.
- We need to know if things worked out (found a solution) or they didn't, and if they didn't try the next number
- If we try all numbers and none of them work in our cell we need to *report back* that things didn't work

Recursive Backtracking

- Problems such as Suduko can be solved using recursive backtracking
- recursive because later versions of the problem are just slightly simpler versions of the original
- backtracking because we may have to try different alternatives

ตัวอย่าง backtracking : Compiler

- Grammar

$$E \rightarrow T \mid T + E$$
$$T \rightarrow int \mid int * T \mid (E)$$

- input stream (int_5)
- ให้ตรวจสอบว่า input นี้ถูกต้องตามหลักไวยากรณ์หรือไม่
- ตัวเริ่มต้นของ Grammar คือ E
- ทดลองกฎต่างๆ ตามลำดับการจัดเรียง

ตัวอย่าง backtracking : Compiler

$E \rightarrow T \mid T + E$

$T \rightarrow int \mid int * T \mid (E)$

E

(int_5)



ตัวอย่าง backtracking : Compiler

$E \rightarrow T \mid T + E$

$T \rightarrow \text{int} \mid \text{int} * T \mid (E)$

E
|
T

(int_5)
↑

ตัวอย่าง backtracking : Compiler

$E \rightarrow T \mid T + E$

$T \rightarrow int \mid int * T \mid (E)$



Mismatch: int is not (!
Backtrack ...

(int_5)
↑

ตัวอย่าง backtracking : Compiler

$E \rightarrow T \mid T + E$

$T \rightarrow int \mid int * T \mid (E)$

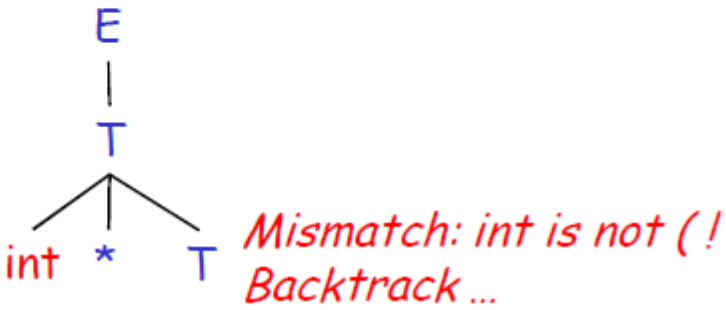


(int_5)

ตัวอย่าง backtracking : Compiler

$E \rightarrow T \mid T + E$

$T \rightarrow int \mid int * T \mid (E)$



(int_5)

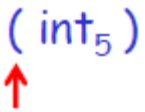
↑

ตัวอย่าง backtracking : Compiler

$E \rightarrow T \mid T + E$

$T \rightarrow int \mid int * T \mid (E)$

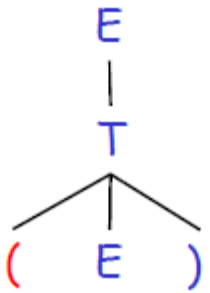




ตัวอย่าง backtracking : Compiler

$E \rightarrow T \mid T + E$

$T \rightarrow int \mid int * T \mid (E)$



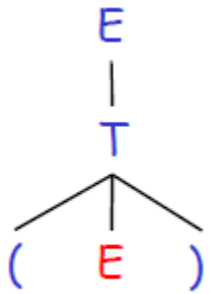
Match! Advance input.

(int_5)

ตัวอย่าง backtracking : Compiler

$E \rightarrow T \mid T + E$

$T \rightarrow int \mid int * T \mid (E)$

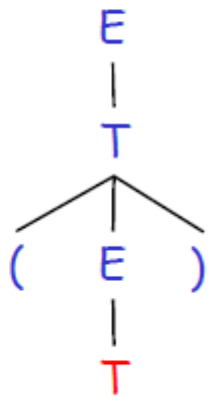


(int_5)

ตัวอย่าง backtracking : Compiler

$E \rightarrow T \mid T + E$

$T \rightarrow \text{int} \mid \text{int} * T \mid (E)$

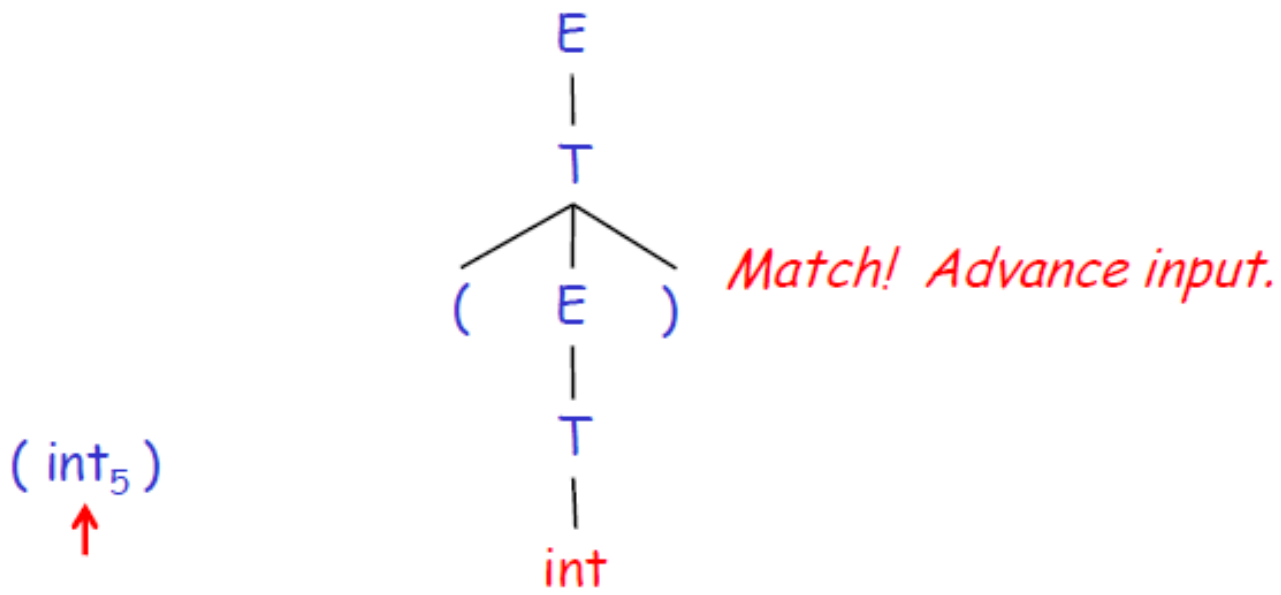


(int_5)

ตัวอย่าง backtracking : Compiler

$E \rightarrow T \mid T + E$

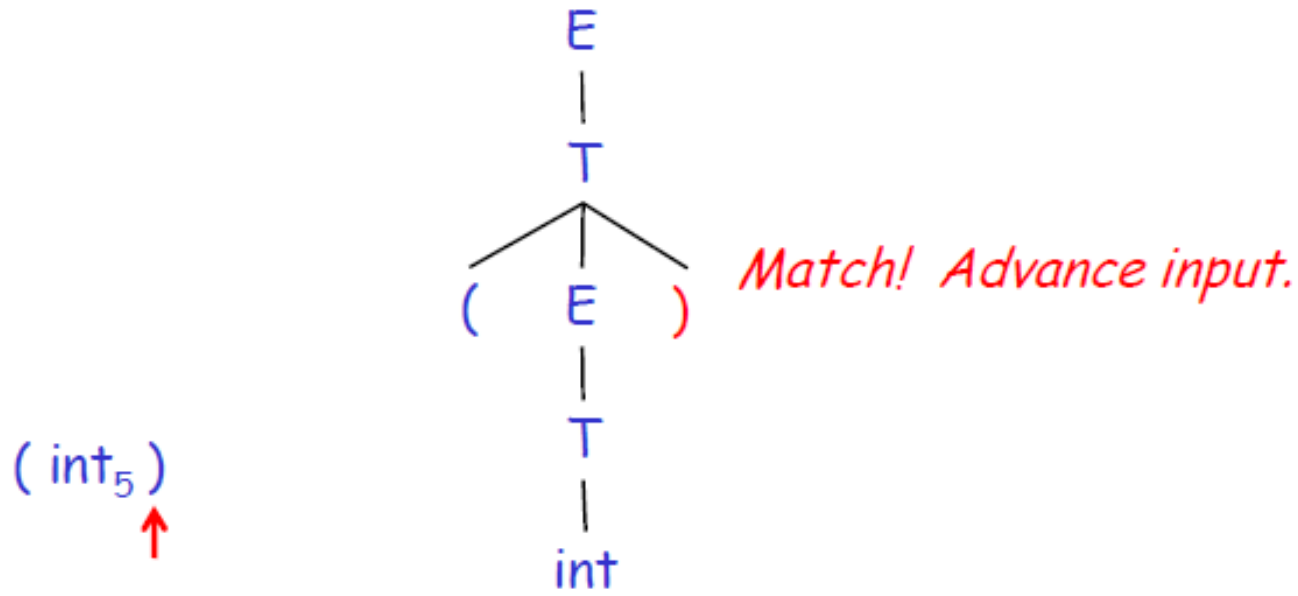
$T \rightarrow int \mid int * T \mid (E)$



ตัวอย่าง backtracking : Compiler

$E \rightarrow T \mid T + E$

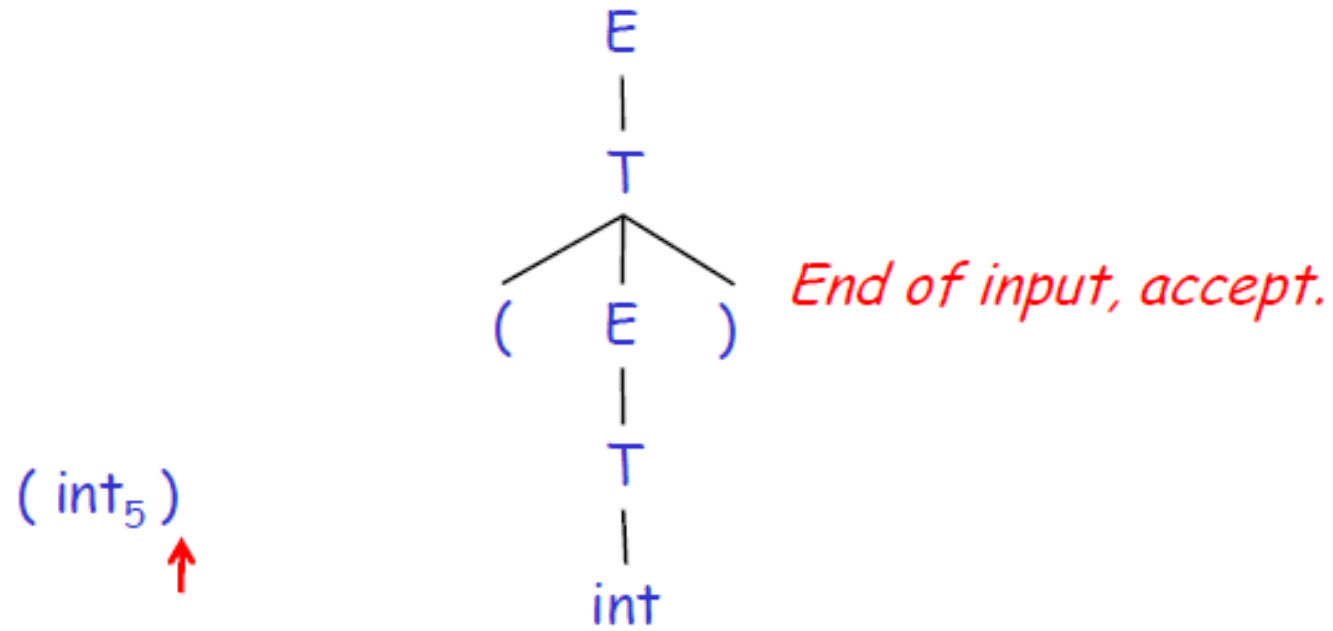
$T \rightarrow int \mid int * T \mid (E)$



ตัวอย่าง backtracking : Compiler

$E \rightarrow T \mid T + E$

$T \rightarrow int \mid int * T \mid (E)$



Top-down parser

$E \rightarrow T \mid T + E$
$T \rightarrow int \mid int * T \mid ($

Backtracking Parser(Recursive Descent Parsing)

```
     void A() {
1)       Choose an A-production, A → X1 X2 ··· Xk;
2)       for ( i = 1 to k ) {
3)           if ( Xi is a nonterminal )
4)               call procedure Xi();
5)           else if ( Xi equals the current input symbol a )
6)               advance the input to the next symbol;
7)           else /* an error has occurred */;
         }
     }
```

Figure 4.13: A typical procedure for a nonterminal in a top-down parser

This algorithm need to be modified for backtracking.

ปัญหา Coloring a map

- ให้ใช้สีไม่เกิน 4 สีในการระบายพื้นที่จังหวัดต่าง ๆ ของประเทศไทย (ทำได้หรือไม่?)
- โดยที่จังหวัดที่อยู่ติดกันจะต้องมีสีที่ต่างกัน
- เราไม่มีข้อมูลอื่นๆ เพิ่มเติม
- ก่อนอื่น ให้นิสิตเปลี่ยนภาพแผนที่เป็น graph



ที่มา http://khtpschool.ning.com/profiles/blogs/asean-contntries-motto-and-map

ให้แปลง map เป็น Graph



2 : 1,3,4

3: 1,2, 4

4: 1,2,3,5,6

5 : 1,4,6

6: 4,5

Download from
Dreamstime.com
This watermarked comp image is for previewing purposes only.
ID 6304838
Indos82 | Dreamstime.com

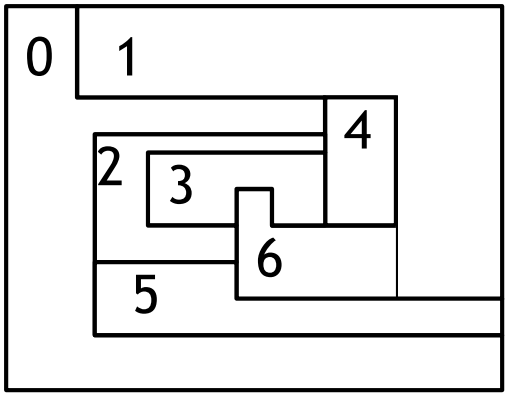
ตัวอย่าง : Map coloring

- The Four Color Theorem states that
  any map on a plane can be colored
  with no more than four colors,
  so that no two countries with a common border
  are the same color
- For most maps, finding a legal coloring is easy
- For some maps, it can be fairly difficult to find a legal coloring
- We will develop a complete Java program to solve this problem

Data structures

- We need a data structure that is easy to work with, and supports:
  - Setting a color for each country
  - For each country, finding all adjacent countries
- We can do this with two arrays
  - An array of "colors", where `mapColors[i]` is the color of the ith country
  - A ragged array of adjacent countries, where `map[i][j]` is the jth country adjacent to country i
    - Example: `map[5][3]==8` means the 3th country adjacent to country 5 is country 8

Creating the map




```
int map[][];

void createMap() {
    map = new int[7][];
    map[0] = new int[] { 1, 4, 2, 5 };
    map[1] = new int[] { 0, 4, 6, 5 };
    map[2] = new int[] { 0, 4, 3, 6, 5 };
    map[3] = new int[] { 2, 4, 6 };
    map[4] = new int[] { 0, 1, 6, 3, 2 };
    map[5] = new int[] { 2, 6, 1, 0 };
    map[6] = new int[] { 2, 3, 4, 1, 5 };
}
```

Setting the initial colors

```
static final int NONE = 0;
static final int RED = 1;
static final int YELLOW = 2;
static final int GREEN = 3;
static final int BLUE = 4;

int mapColors[] = { NONE, NONE, NONE, NONE,
                    NONE, NONE, NONE };
```

The main program

(The name of the enclosing class is ColoredMap)

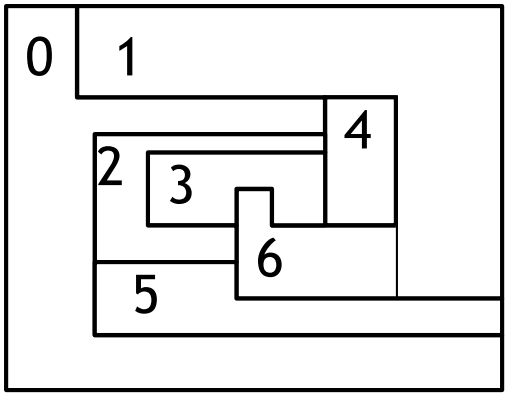
```
public static void main(String args[]) {
    ColoredMap m = new ColoredMap();
    m.createMap();
    boolean result = m.explore(0, RED);
    System.out.println(result);
    m.printMap();
}
```

The backtracking method

```
boolean explore(int country, int color) {
    if (country >= map.length) return true;
    if (okToColor(country, color)) {
        mapColors[country] = color;
        for (int i = RED; i <= BLUE; i++) {
            if (explore(country + 1, i)) return true;
        }
    }
    return false;
}
```

Checking if a color can be used

```
boolean okToColor(int country, int color) {
    for (int i = 0; i < map[country].length; i++)
        int ithAdjCountry = map[country][i];
        if (mapColors[ithAdjCountry] == color) {
            return false;
        }
    }
    return true;
}
```



Printing the results

```
void printMap() {
    for (int i = 0; i < mapColors.length; i++) {
        System.out.print("map[" + i + "] is ");
        switch (mapColors[i]) {
            case NONE:   System.out.println("none");   break;
            case RED:    System.out.println("red");    break;
            case YELLOW: System.out.println("yellow"); break;
            case GREEN:  System.out.println("green");  break;
            case BLUE:   System.out.println("blue");   break;
        }
    }
}
```

Recap

- We went through all the countries recursively, starting with country zero
- At each country we had to decide a color
  - It had to be different from all adjacent countries
  - If we could not find a legal color, we reported failure
  - If we could find a color, we used it and recurred with the next country
  - If we ran out of countries (colored them all), we reported success
- When we returned from the topmost call, we were done

Recursive Backtracking

Pseudo code for recursive backtracking algorithms

If at a solution, report success
for(every possible choice from current state / node)
    Make that choice and take one step along path
    Use recursion to solve the problem for the new node / state
    If the recursive call succeeds,
        report the success to the next lower level
    Back out of the current choice to restore the state
        at the beginning of the loop.
Report failure



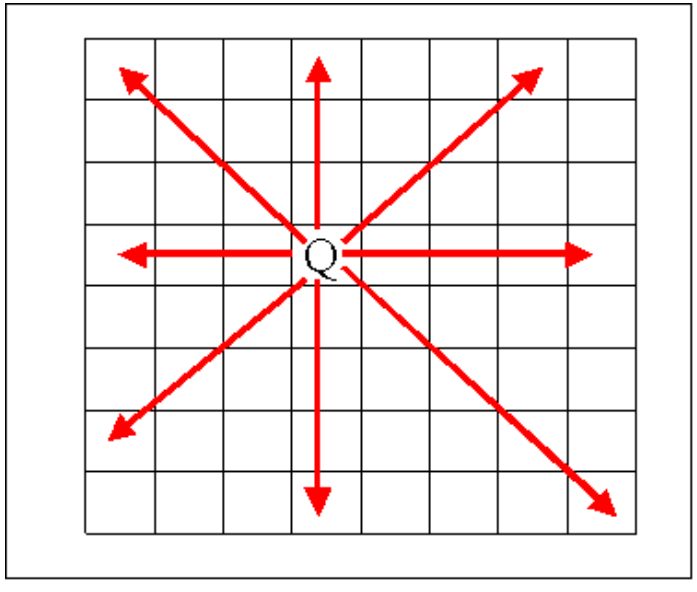
The 8 Queens Problem

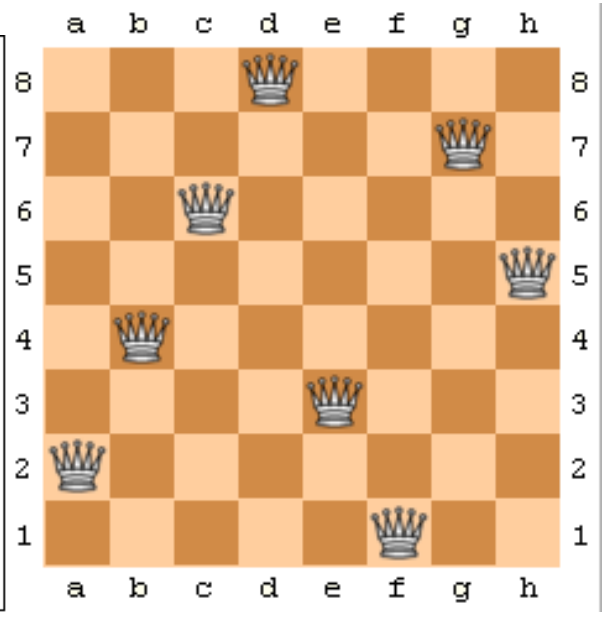


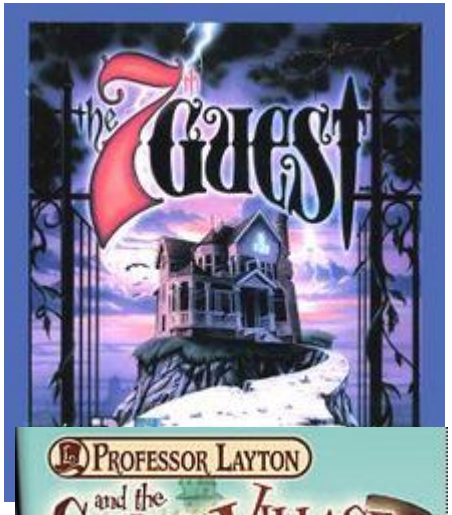


The 8 Queens Problem

- A classic chess puzzle
  - Place 8 queen pieces on a chess board so that none of them can attack one another









The N Queens Problem

- Place N Queens on an N by N chessboard so that none of them can attack each other
- Number of possible placements?
- In 8 x 8

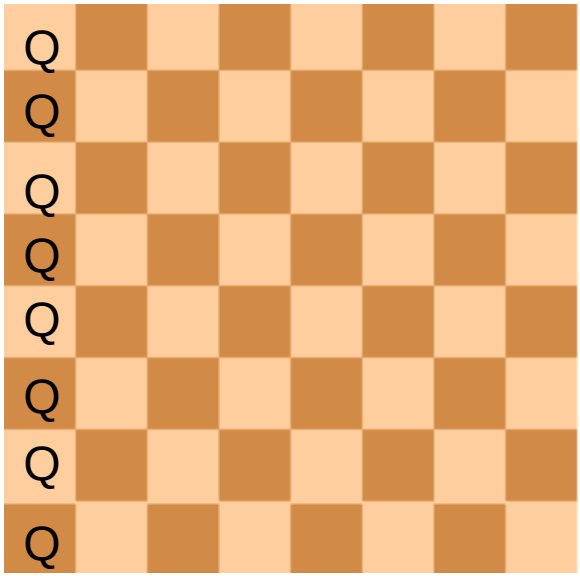
  64 * 63 * 62 * 61 * 60 * 59 * 58 * 57
  = 178,462, 987, 637, 760 / 8!
  = 4,426,165,368

  n choose k $$\binom{n}{k} = \frac{n \cdot (n-1) \cdots (n-k+1)}{k \cdot (k-1) \cdots 1} = \frac{n!}{k!(n-k)!} \quad \text{if } 0 \leq k \leq n \qquad (1)$$

  – How many ways can you choose k things from a set of n items?
  – In this case there are 64 squares and we want to choose 8 of them to put queens on

Reducing the Search Space

- The previous calculation includes set ups like this one



- Includes lots of set ups with multiple queens in the same column
- How many queens can there be in one column?
- Number of set ups
  8 * 8 * 8 * 8 * 8 * 8 * 8 * 8 = 16,777,216
- We have reduced search space by two orders of magnitude by applying some logic

A Solution to 8 Queens

- If number of queens is fixed and I realize there can't be more than one queen per column I can iterate through the rows for each column

```
for(int r0 = 0; r0 < 8; r0++){
       board[r0][0] = 'q';
        for(int r1 = 0; r1 < 8; r1++){
              board[r1][1] = 'q';
              for(int r2 = 0; r2 < 8; r2++){
                     board[r2][2] = 'q';
                     // a little later
                     for(int r7 = 0; r7 < 8; r7++){
                            board[r7][7] = 'q';
                            if( queensAreSafe(board) )
                                   printSolution(board);
                            board[r7][7] = ' '; //pick up queen
                     }
                     board[r6][6] = ' '; // pick up queen
```

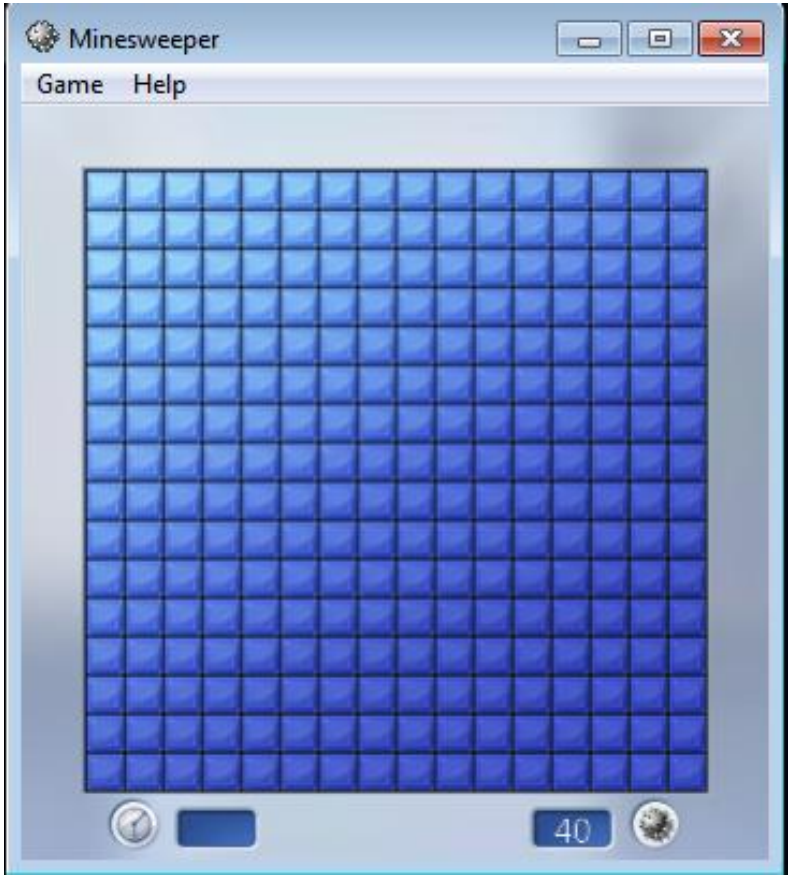
N Queens

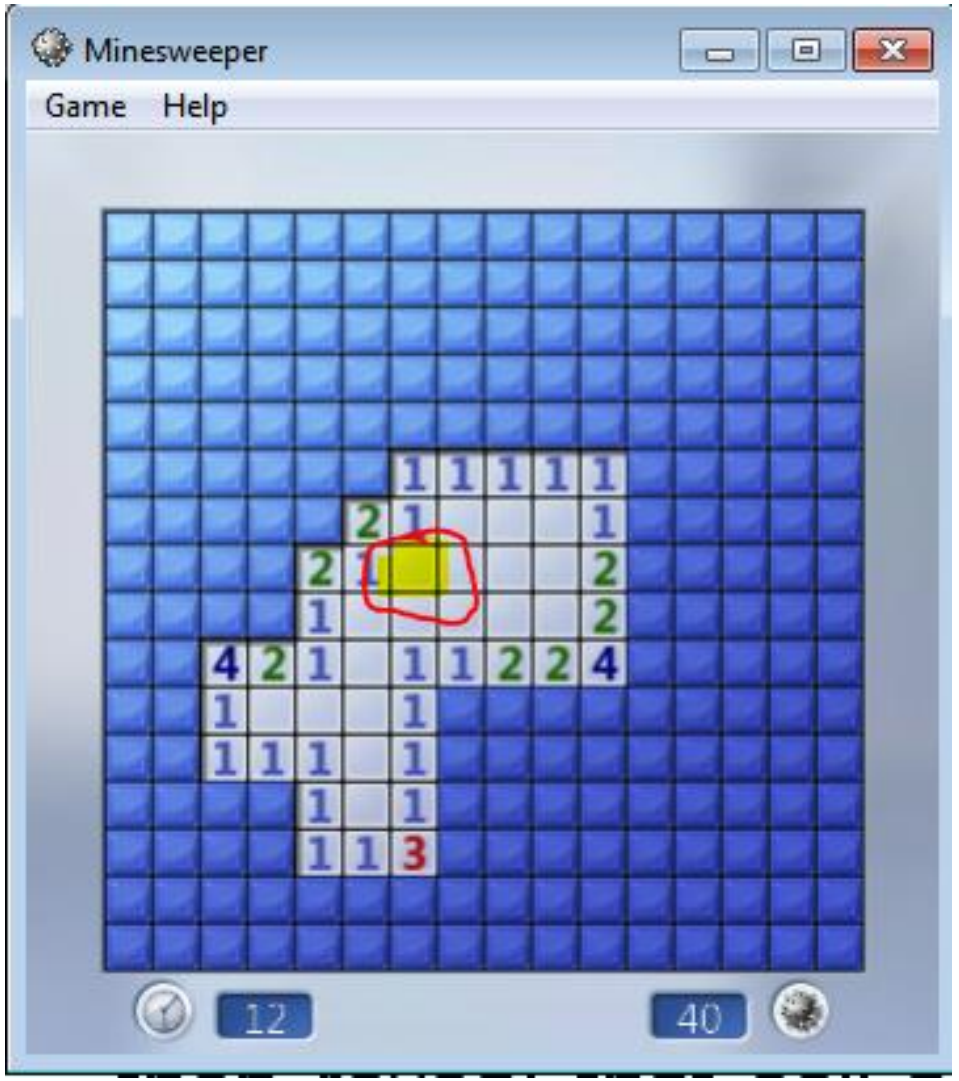
- The *problem* with N queens is
  you don't know how many for loops to write.
- Do the problem recursively
- Write recursive code with class and demo
  - show backtracking with breakpoint and debugging option

Recursive Backtracking

- You must practice!!!
- Learn to recognize problems
  that fit the pattern
- Is a *kickoff* method needed?
- All solutions or a solution?
- Reporting results and acting on results

Minesweeper





Minesweeper Reveal Algorithm

- Minesweeper
- click a cell
  - if bomb game over
  - if cell that has 1 or more bombs on border
    then reveal the number of bombs that border cell
  - if a cell that has 0 bombs on border
    then reveal that cell as a blank and click on the 8 surrounding cells

8-puzzle

Initial State

1	2	3
8		4
7	6	5

Goal State

2	8	1
	4	3
7	6	5